Introdução à Computação
alocação dinâmica de memória

Alexandre Rademaker

alocação dinâmica de memória I

Vimos como declarar ponteiros e **apontá-los** para variáveis já existentes. Isto é, preencher seu valor com endereços já alocados na memória.

Em vários problemas que vimos até agora, alocamos trechos de memória além do necessário exatamente por não saber a quantidade de memória exatamente necessária durante a compilação.

Em alguns casos a quantidade de memória necessária não é conhecida no momento da compilação mas apenas no tempo de execução. (ex: lendo um arquivo)

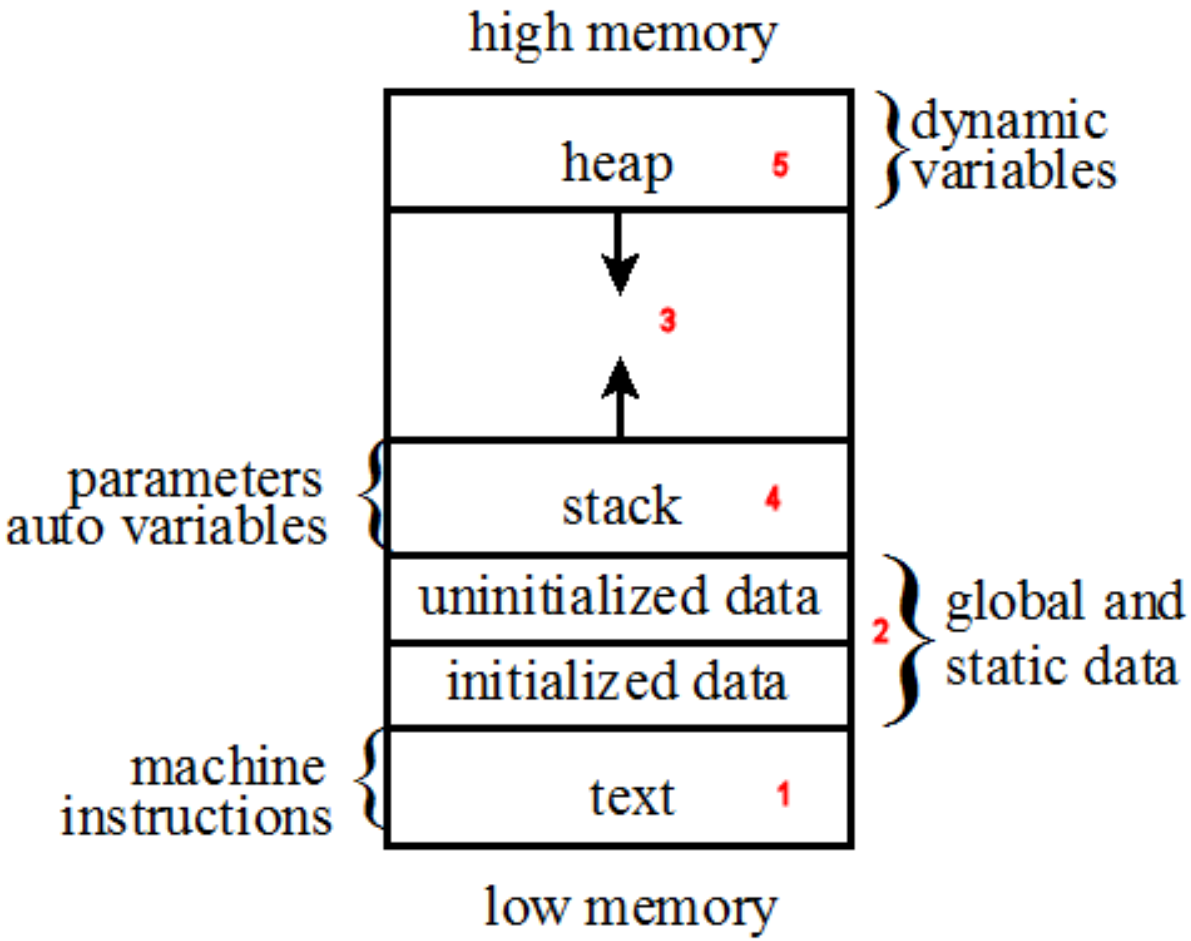
alocação dinâmica de memória II

Podemos atribuir novos endereços de memória para ponteiros também, endereços alocados de forma dinâmica, durante a execução do programa.

Memória alocada de forma dinâmica vem de um espaço conhecido como **heap**.

Até agora, só usávamos memória da área conhecida como **stack**.

alocação dinâmica de memória III



alocação dinâmica de memória IV

Para alocar memória de maneira dinâmica, usamos a função `malloc()` da biblioteca C standard library, passando para ela o número de **bytes** necessários.

A função `malloc()` retorna um ponteiro para a nova região alocada. Se `malloc()` não conseguir alocar a memória solicitada, irá retornar `NULL`.

alocação dinâmica de memória V

```
1 // alocacao estatica
2 int x;
3
4 // alocacao dinamica
5 int *px = malloc(4);
```

alocação dinâmica de memória VI

```
1 // alocacao estatica
2 int x;
3
4 // alocacao dinamica
5 int *px = malloc(sizeof(int));
```

O operador `sizeof` retorna o tamanho que um tipo que recebe como argumento. Também podemos passar uma variável ou expressão para o `sizeof` para obter o tamanho ocupado por ela.

alocação dinâmica de memória VII

```
1 // lendo um valor do usuario
2 int x = get_int_from_user();
3
4 // array de x elementos float na stack
5 float arr1[x];
6
7 // array de x elementos float no heap
8 float* arr2 = malloc(x * sizeof(float));
```

alocação dinâmica de memória VIII

Mas temos um problema!

Memória alocada de forma dinâmica não é controlada pelo sistema e não é automaticamente liberada quando o escopo onde ela foi declarada termina.

Se a memória não for liberada, temos o que chamamos de **memory leak**. Memória perdida que o sistema não sabe que poderia usar, logo a performace do sistema pode degradar.

O programador deve, quando terminar de usar a memória, liberá-la com a função `free()`.

alocação dinâmica de memória IX

```
1 // alocacao dinamica
2 int *px = malloc(sizeof(int));
3
4 // faz alguma coisa
5
6 // liberando
7 free(px);
```

alocação dinâmica de memória X

Regras importantes

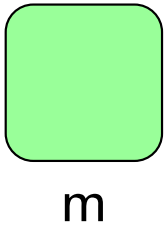
- todo bloco de memória alocado com `malloc()` deve ser liberado com `free()`.
- apenas memória alocada com o `malloc()` pode ser liberada com `free()`.
- não tente liberar o mesmo bloco de memória duas vezes.

alocação dinâmica I

```
int m;
```

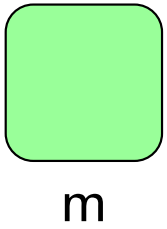
alocação dinâmica II

```
int m;
```



alocação dinâmica III

```
int m;
int* a;
```



alocação dinâmica IV

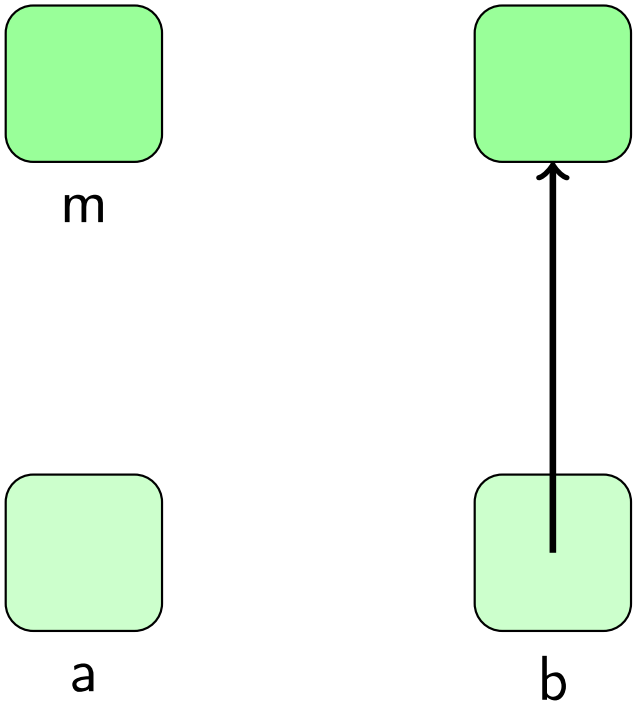
```
int m;
int* a;
```

alocação dinâmica V

```
int m;
int* a;
int* b = malloc(sizeof(int));
```

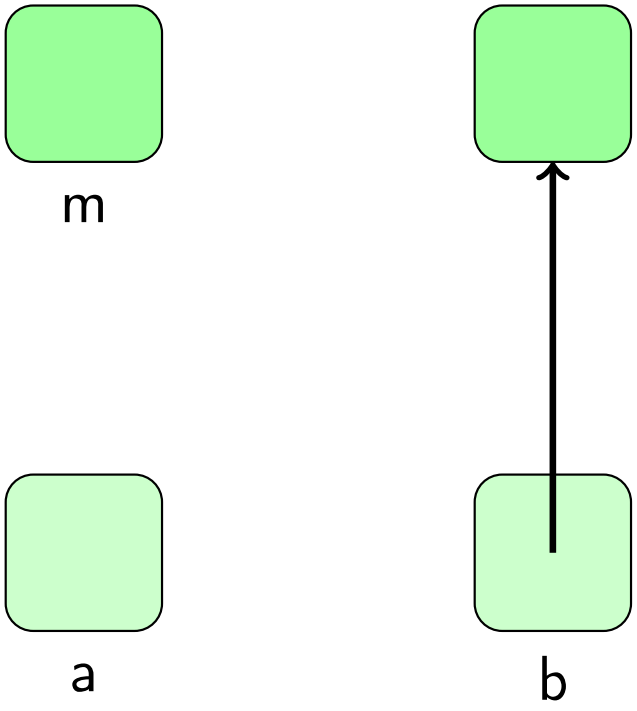
alocação dinâmica VI

```
int m;
int* a;
int* b = malloc(sizeof(int));
```



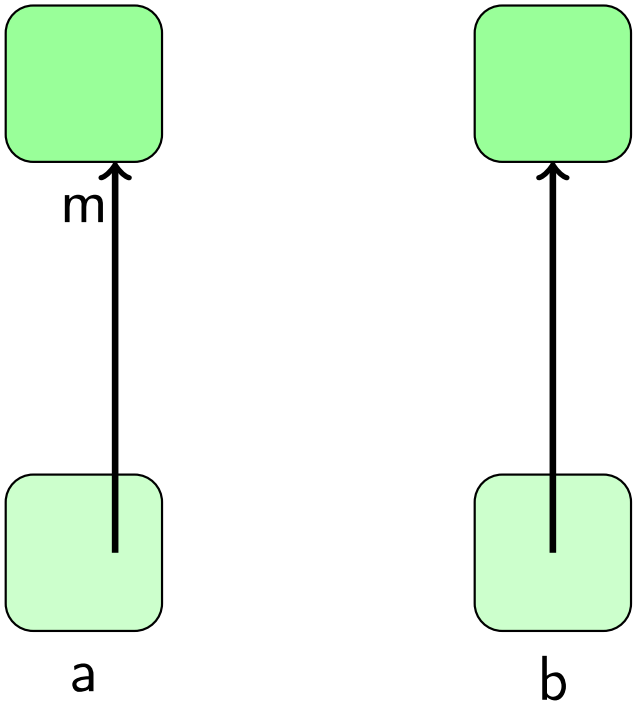
alocação dinâmica VII

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
```



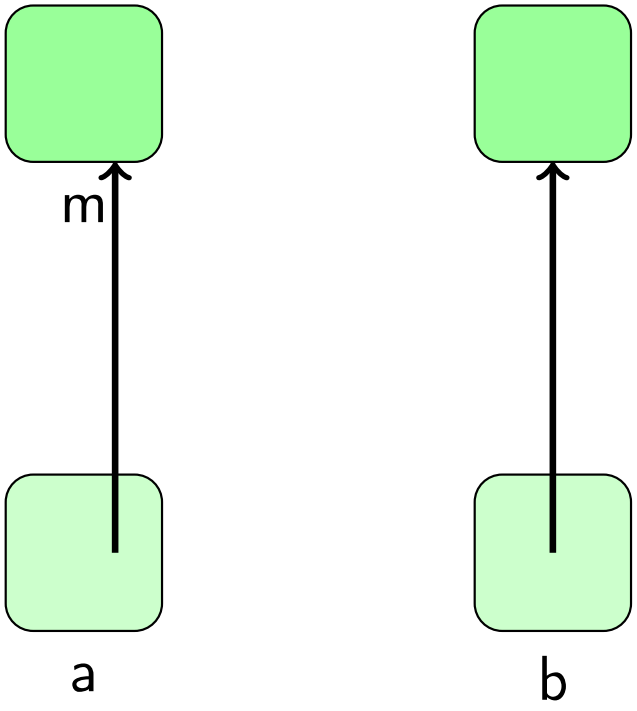
alocação dinâmica VIII

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
```



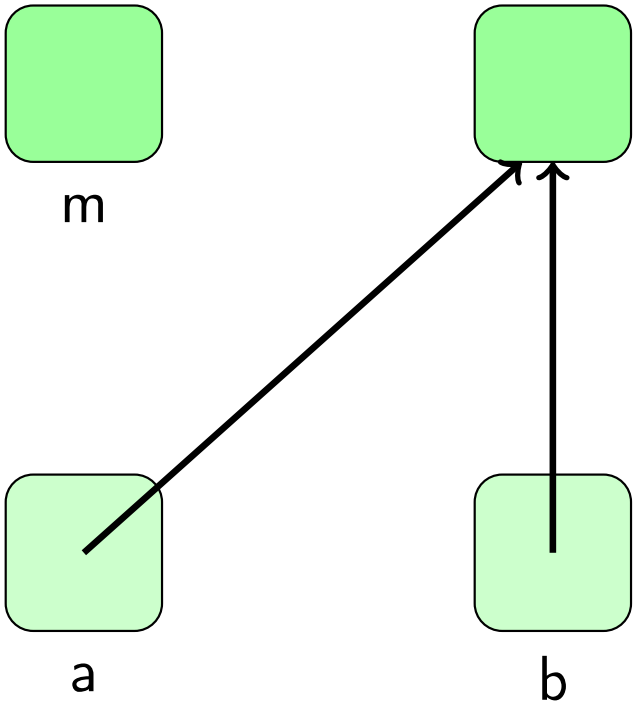
alocação dinâmica IX

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
```



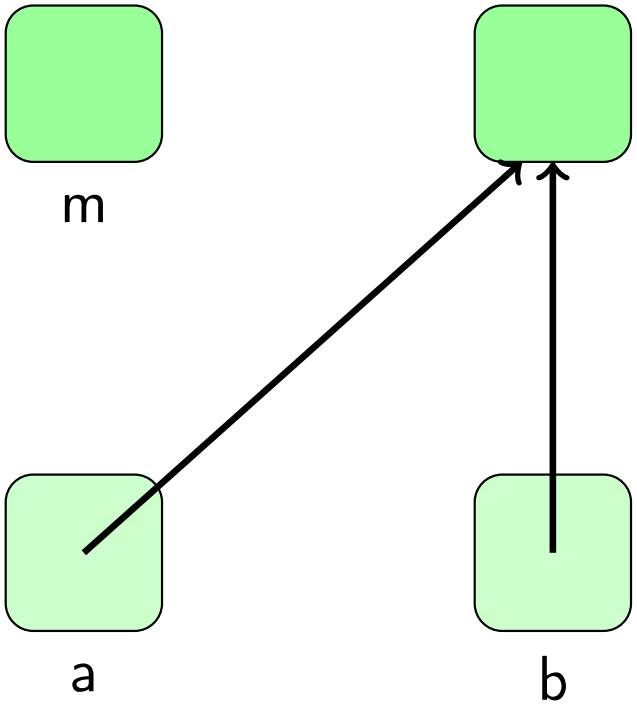
alocação dinâmica X

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
```



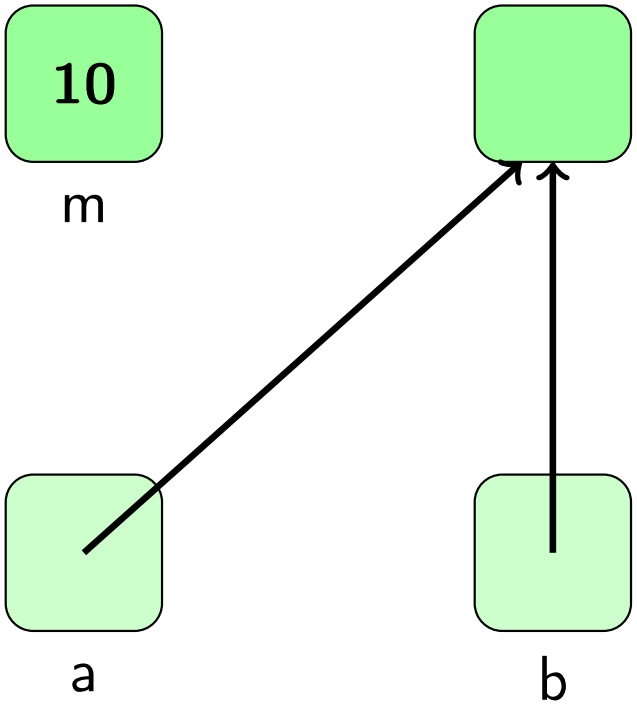
alocação dinâmica XI

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
```



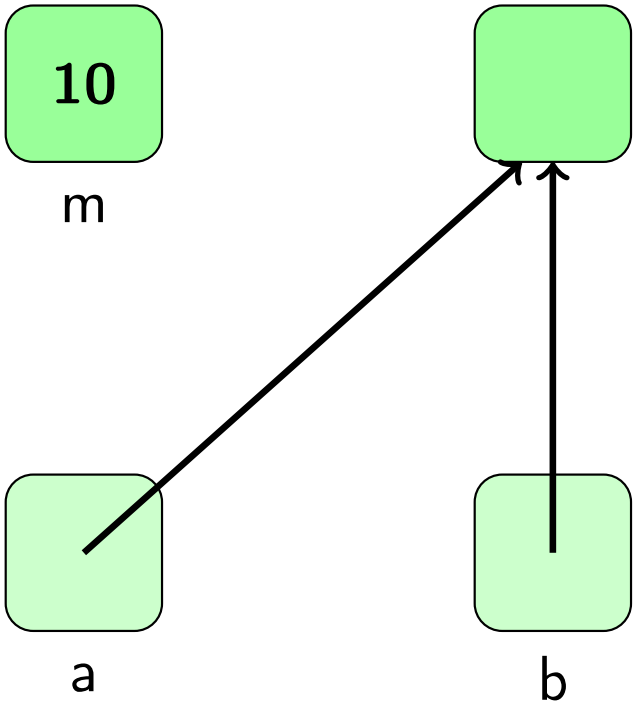
alocação dinâmica XII

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
```



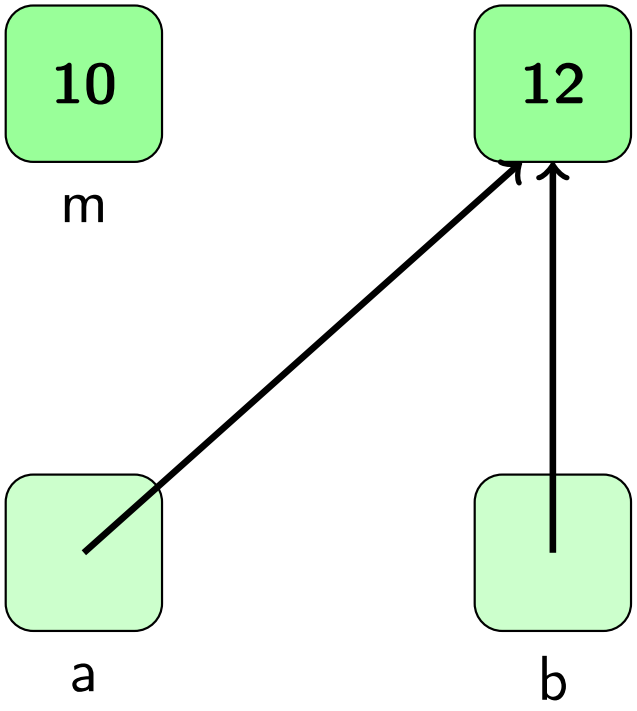
alocação dinâmica XIII

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
*b = m + 2;
```



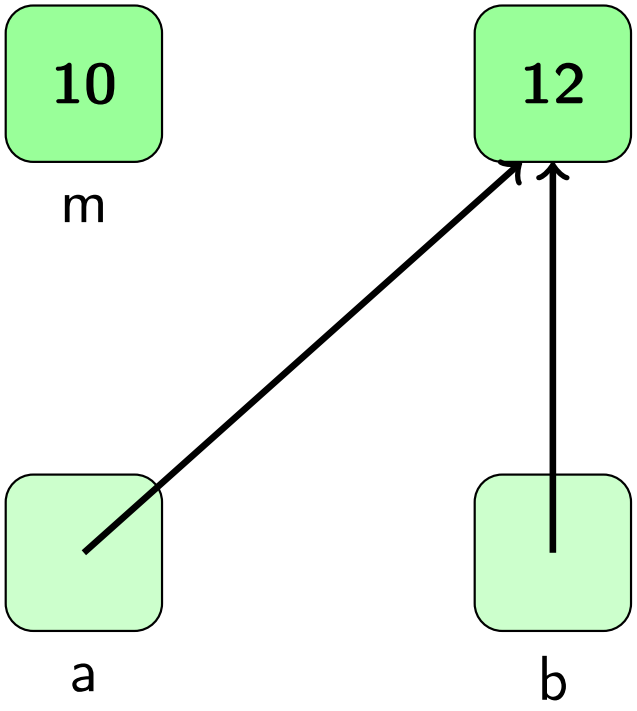
alocação dinâmica XIV

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
*b = m + 2;
```



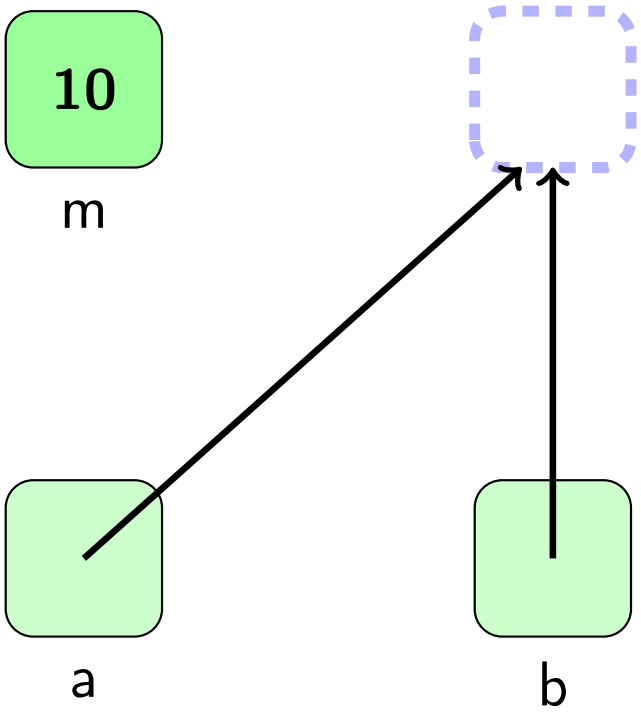
alocação dinâmica XV

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
*b = m + 2;
free(b);
```



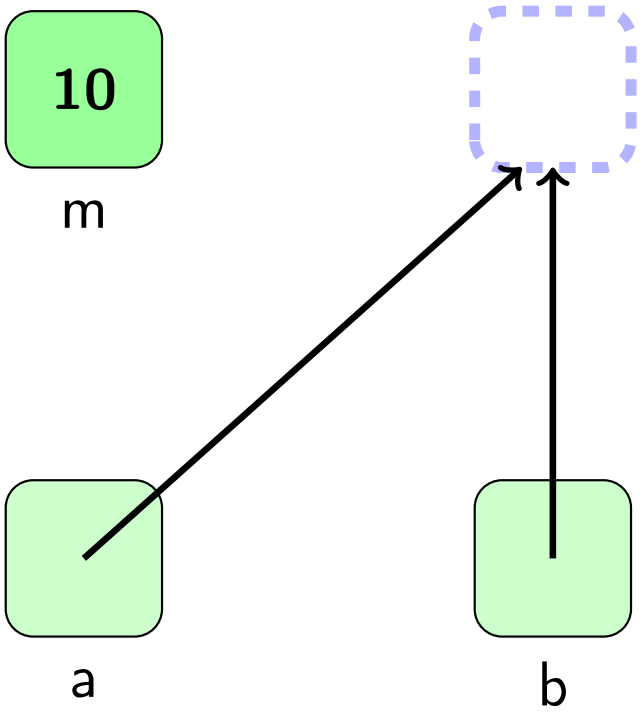
alocação dinâmica XVI

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
*b = m + 2;
free(b);
```



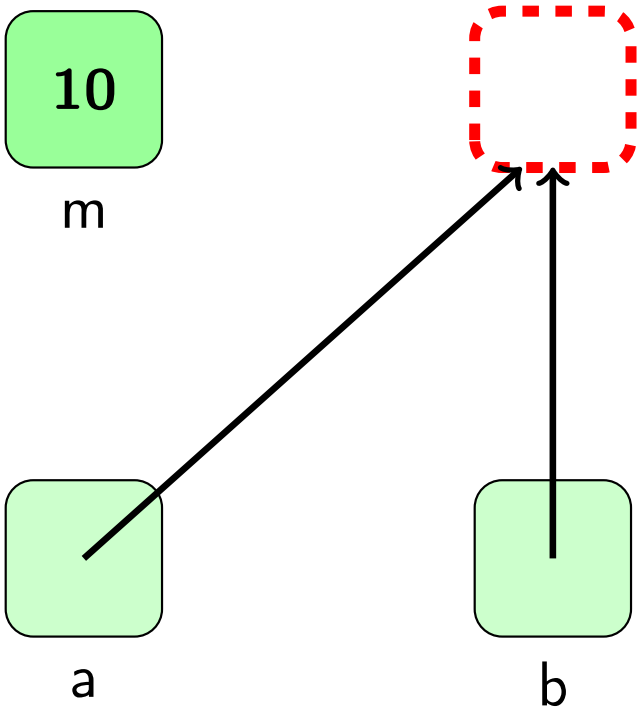
alocação dinâmica XVII

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
*b = m + 2;
free(b);
*a = 11;
```



alocação dinâmica XVIII

```
int m;
int* a;
int* b = malloc(sizeof(int));
a = &m;
a = b;
m = 10;
*b = m + 2;
free(b);
*a = 11;
```



copia de string I

Voltando ao problema de copiar uma string.

Precisamos na verdade, criar uma nova região na memória para armazenar os bytes necessários para a cópia.

copia de string II

src/copy0.c

```
1  #include <ctype.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  int main(void) {
6    char s[] = "hi!";
7    char *t = s;
8
9    // capitalize first letter in string
10   if (strlen(s) > 0) {
11     s[0] = toupper(s[0]);
12   }
13
14   printf("s: %s\n", s);
15   printf("t: %s\n", t);
16   return 0;
17 }
```

copia de string III

src/copy1.c

```c
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
  char s[] = "hi!";
  char *t = malloc(strlen(s) + 1);

  for (int i = 0, n = strlen(s); i <= n; i++)
    t[i] = s[i];
  t[0] = toupper(t[0]);

  printf("s: %s\nt: %s\n", s, t);
  free(t);
  return 0;
}
```

copia de string IV

src/copy2.c

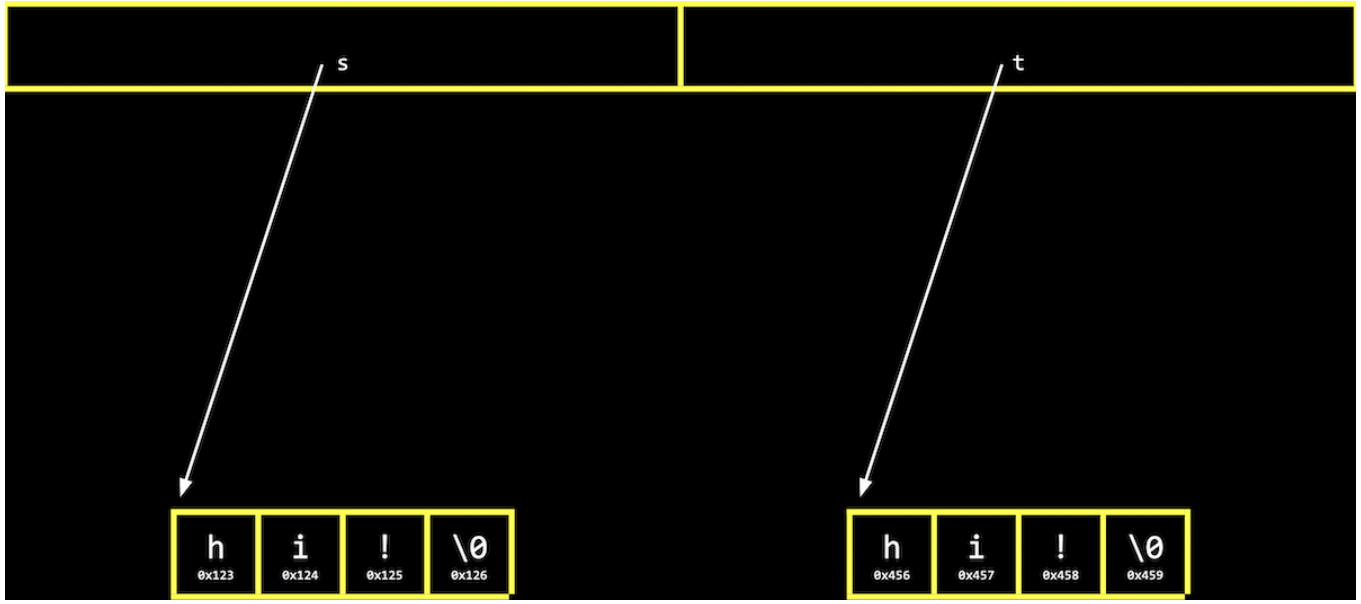
```c
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
  char s[] = "hi!";
  char *t = malloc(strlen(s) + 1);
  // char *t = NULL;

  strcpy(t, s);
  t[0] = toupper(t[0]);

  printf("s: %s\nt: %s\n", s, t);
  free(t);
  return 0;
}
```

copia de string V



Alocamos um espaço de memória `0x456` e fizemos `t` apontar para ele. Copiamos os bytes entre as regiões com o **strcpy**.